UNIVERZITA KARLOVA

Přírodovědecká fakulta

Katedra aplikované geoinformatiky a kartografie

Studijní program:

Geoinformatika, kartografie a dálkový průzkum Země

Jakub Fuchsig, Šimon Kohout

Generalizace budov LOD0

Ústí nad Labem, Zákupy 2026

1 Zadání

Vstup: množina budov $B = \{B_i\}_{i=1}^{n}$, budova $B_i = \{P_{i,j}\}_{j=1}^{m}$.

Výstup: $G(B_i)$.

Ze souboru načtěte vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LOD0. Pro tyto účely použijte vhodnou datovou sadu, např. ZABAGED, testování proveďte nad třemi datovými sadami (historické centrum města, sídliště, izolovaná zástavba).

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle,
- PCA.

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LOD0 nahraďte obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a porovnejte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budov dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

2 Zpracované bonusové úlohy

Pro tuto úlohu byly zpracovány níže vypsané úlohy společně s jejich ohodnocením.

Krok	**Hodnocení**
Generalizace budov metodami Minimum Area Enclosing Rectangle a PCA.	15b
Generalizace budov metodou Longest Edge.	+5b
Generalizace budov metodou Wall Average.	+8b
Generalizace budov metodou Weighted Bisector.	+10b
Ošetření singulárních případů při generování konvexní obálky.	+2b
*Načtení vstupních dat ze *.shp.*	+10b
Celkem splněno:	**50b**

3 Úvod

Různá měřítka map zobrazují různé oblasti s odlišnými přiblíženími. Důležitým krokem při změně měřítka je rozhodnutí, jak budou některé mapové prvky zobrazeny v mapě. Tento proces se nazývá generalizace a je jedním z nejdůležitějších kroků při tvorbě mapového díla. Cílem tohoto procesu je zachování přibližného tvaru, orientace či plochy prvku.

Pro tuto úlohu byly generalizovány budovy do podoby detailu LOD0, tedy původní půdorys budovy se nahradí jednoduchým geometrickým tvarem, nejčastěji obdélníkem. Proces generalizace byl prováděn metodami, které se snaží zachovat orientaci, plochu a velikost původní budovy. Použitými metodami jsou *Minimum Area Enclosing Rectangle (MBR)*, *Principal Component Analysis (PCA)*, *Longest Edge*, *Wall Average* a *Weighted Bisector*, které tyto požadavky splňují, přestože každá metoda přistupuje k určování, zejm. hlavních směrů, jinak.

Pro testování na reálných datech byly metody aplikovány na třech typech zástavby:

- **historické centrum města** - pro tuto zástavbu bylo zvoleno centrum města Česká Lípa
- **sídliště** - vzorkem této zástavby se stalo sídliště Severní terasa v Ústí nad Labem
- **izolovaná zástavba** - jedná se o budovy, zejm. rodinné domky, z vesnice Božíkov u města Zákupy

4 Konvexní obálka

Konvexní obálka (ang. *convex hull - ch*) je jedna z nejpoužívanějších geometrických struktur nejen kartografické generalizace. Jedná se o konvexní mnohoúhelník, který obsahuje veškeré body a jehož plocha je nejmenší možná. Tuto obálku lze často využít např. ke konstrukci *Minimum Bounding Rectangle (MBR)*, který se právě využívá v kartografické generalizaci pro zjištění tvaru a orientace budov.

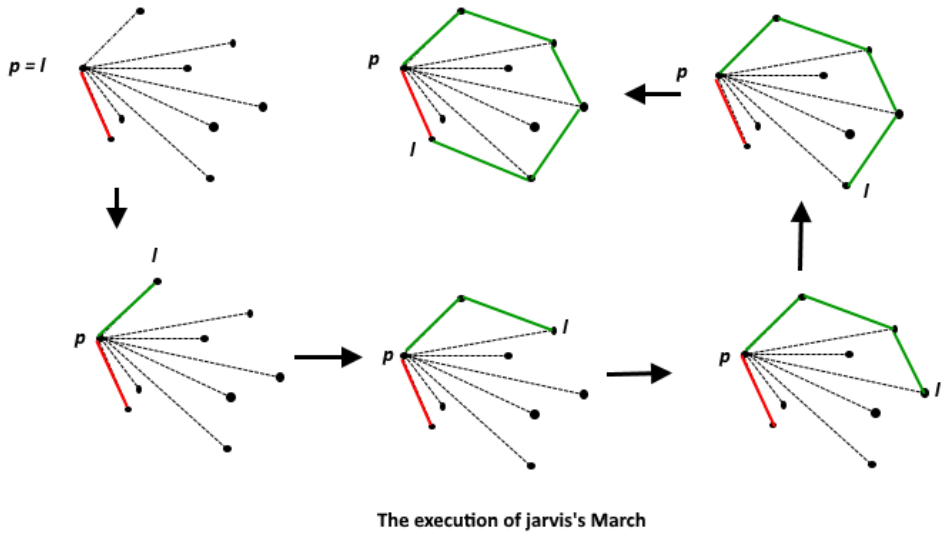
4.1 Konstrukce konvexní obálky

Ke konstrukci tohoto konvexního mnohouhelníku lze přistupovat vícero metodami. Nejčastěji je možné se setkat s metodou ***Jarvis Scan***, který byl použit v tomto úkolu, a s metodou ***Graham Scan***.

4.1.1 Jarvis Scan

Jedná se o velmi intuitivní přístup konstrukce konvexní obálky. Metoda funguje na principu maximalizace vnitřního úhlu k hledaným bodům. V každém kroku je tedy hledán nový bod, který svírá s aktuální hranou největší úhel - tudíž se jedná o hranici množiny. Důležitým předpokladem je, že metoda je iniciována v bodě s nejmenší Y-ovou (případně X-ovou) souřadnicí. Nevýhodou je jeho časová složitost ve velkých množinách, jelikož prochází v každém kroku všechny body.

Níže je zobrazen princip algoritmu *Jarvis Scan*.



Obrázek 1: Princip algoritmu Jarvis Scan (OpenGenusIQ 2026)

4.1.2 Graham Scan

Tento algoritmus nebyl v této práci využit, přestože je to efektivnější přístup. Jeho nevýhodou však je, že ho není možné použít pro 3D data. Pro tuto metodu je opět nutné zvolení počátečního bodu, od kterého jsou následně počítány úhly ke zbývajícím bodům a ty se následně seřadí. Body se následně cyklicky procházejí a pokud spadají do **levé** poloroviny, stávají se součástí obálky, pokud náleží **pravé** polorovině, jsou z obálky odebrány.

5 Použité algoritmy

5.1 Algoritmus *Minimum Area Enclosing Rectangle*

Tento algoritmus je založen na mětodě hledání opsaného obdélníku s minimální plochou. Vychází z teoretického předpokladu, že alespoň jedna strana takového obdélníku musí být kolineární s hranou konvexní obálky objektu. Proces zahrnuje nejprve konstrukci konstrukci konvexní obálky, poté natočení obálky tak, aby jejejí hrana byla rovnoběžná s osou x (úhel $-sigma$). Dále je nutné najít její minimální a maximální souřadnice x a y, z nichž se dále vytvoří *min-max box*. Po výpočtu plochy a nalezení nejmenšího boxu je posléze zpět natočet o úhel *sigma* do původní orientace (Bayer 2026).

V naší implementaci je proces nalezení tohoto obdélníku obsažen ve funkci `createMBR` a jeho finální úprava ve funkci `simplifyBuildingMBR`. Metoda využívá následující kroky:

1. **Konstrukce konvexní obálky:** Z polygonu původní budovy se pomocí funkce `createCH` (Jarvis Scan) vytvoří konvexní obálka `ch`.
2. **Inicializace minima:** Pro počáteční porovnávání se nad obálkou `ch` vytvoří základní min-max box `mmb_min` a pomocí funkce `createMMB` se vypočte jeho plocha `area_min`.
3. **Iterace a rotace obálky:** Algoritmus v cyklu prochází všechny hrany obálky. Pro každou hranu se z rozdílu souřadnic koncových bodů (`dx`, `dy`) vypočte její směrový úhel `sigma` (pomocí funkce `atan2`). Obálka se následně pomocí `rotatePolygon` natočí o úhel `-sigma` tak, aby daná hrana ležela rovnoběžně s osou x (vznikne rotovaná obálka `ch_r`).
4. **Určení aktuálního min-max boxu:** Nad rotovanou obálkou `ch_r` se opět zavolá funkce `createMMB`, která nalezne aktuální ohraničující obdélník `mmb` a vypočte jeho plochu `area`.
5. **Aktualizace minima:** Je-li aktuální plocha `area` menší než dosud nejmenší známá plocha `area_min`, hodnoty se aktualizují: uloží se nová minimální plocha, `mmb_min = mmb` a zapamatuje se nejlepší úhel natočení `sigma_min = sigma`.
6. **Zpětná rotace a upravení velikosti:** Nejmenší nalezený min-max box `mmb_min` se natočí zpět o úhel `sigma_min`, čímž vznikne obdélník `mbr` orientovaný ve směru budovy. Finálně se zavolá funkce `resizeRectangle`, která velikost tohoto obdélníku od jeho středu proporčně upraví (`mbr_res`) tak, aby jeho výsledná plocha přesně odpovídala ploše původní budovy.

5.2 Algoritmus *Principal Component Analysis*

Metoda PCA určuje hlavní směry budovy na základě rozptylu jejích bodů. Matematickým základem je singulární rozklad (SVD) kovarianční matice souřadnic bodů budovy. Vlastní vektory získané tímto rozkladem definují osy, ve kterých má objekt největší a nejmenší variabilitu, což odpovídá jeho hlavním směrům. Metoda je velmi rychlá, avšak může vykazovat nižší efektivitu u pravidelných objektů, jako je čtverec nebo kružnice (Bayer 2026).

V naší implementaci je proces generalizace metodou PCA obsažen ve funkci `simplifyBuildingPCA`. Metoda využívá následující kroky:

1. **Zisk souřadnic:** Z polygonu původní budovy se iterací přes všechny její lomové body oddělí souřadnice do samostatných seznamů `X` a `Y`. Z těchto seznamů se následně pomocí knihovny *NumPy* vytvoří dvourozměrná matice souřadnic `A`.

2. **Výpočet kovarianční matice:** Z matice `A` se pomocí funkce `np.cov` vypočítá kovarianční matice `C`.

3. **Dekompozice matice (SVD):** Na kovarianční matici `C` je aplikován rozklad singulárních hodnot pomocí funkce `np2.svd`. Operace vrací tři matice: `U`, `S` a `V`, kde matice `V` představuje směrové vektory hlavních komponent.

4. **Určení hlavního směru:** K získání hlavního úhlu budovy `sigma` se použije první vektor z matice `V` (prvky `V[0][0]` a `V[0][1]`), který reprezentuje směr s největším rozptylem a aplikací funkce `atan2`.

5. **Rotace a min-max box:** Polygon původní budovy se pomocí funkce `rotatePolygon` natočí o úhel `-sigma` tak, aby byl jeho hlavní směr rovnoběžný s osou x (vznikne rotovaná budova `build_rot`). Nad tímto polygonem se následně zavolá funkce `createMMB`, která vytvoří ohraničující *min-max box* `mmb`.

6. **Zpětná rotace a upravení velikosti:** Min-max box `mmb` se pomocí `rotatePolygon` natočí zpět o úhel `sigma`, čímž vznikne obdélník `mbr` orientovaný podle hlavního směru budovy. Finálně se využije funkce `resizeRectangle`, která upraví velikost tohoto obdélníku(`mbr_res`) tak, aby jeho plocha odpovídala ploše původní budovy.

5.3 Algoritmus *Longest Edge*

Tato metoda z předpokládá, že nejdelší dostupná hrana polygonu udává primární směr celé budovy. Algoritmus tedy prochází všechny stěny, nalezne

tu s maximální délkou a vypočítá její úhel. Získaná orientace se následně využije pro pootočení původního polygonu tak, aby se kolem něj dal zkonstruovat obdélník s minimální plochou (min-max box). Ačkoliv je metoda algoritmicky i výpočetně velmi jednoduchá, u složitějších nepravidelných tvarů budov nemusí vždy nejdelší hrana reprezentovat kartograficky nejvhodnější celkový směr (Bayer 2026).

V naší implementaci je tento postup obsažen ve funkci `simplifyBuildingLongestEdge`. Metoda využívá následující kroky:

1. **Procházení hran:** Pomocí cyklu se iteruje přes všechny hrany polygonu budovy. Pro každou hranu se určí rozdíl souřadnic jejích koncových bodů (`dx`, `dy`) a pomocí Pythagorovy věty se vypočítá její délka `length`.
2. **Nalezení maxima a určení směru:** Pokud je vypočtená délka hrany větší než dosavadní zaznamenané maximum (`max_length`), hodnoty se aktualizují. Nová délka se uloží jako největší a pomocí funkce `atan2` se z rozdílů souřadnic (`dy`, `dx`) vypočítá její směrový úhel, který se uloží do proměnné `best_angle`.
3. **Opakující se kroky naší generalizace:** Jakmile je iterace dokončena a je určen hlavní úhel budovy (`best_angle`), provedou se výše zmíněné kroky pro vytvoření a úpravu obdélníku, které jsou identické pro všechny implementované metody:
   - Pomocí `rotatePolygon` se budova natočí o úhel `-best_angle`.
   - Funkcí `createMMB` se nad natočenou budovou vytvoří *min-max box* (`mmb`).
   - Tento box se natočí zpět o úhel `best_angle` do reálné orientace (`mbr`).
   - Funkce `resizeRectangle` proporčně upraví velikost obdélníku k těžišti tak, aby jeho plocha odpovídala původní budově (`mbr_res`).

5.4 Algoritmus *Wall Average*

Alternativou k metodě nejdelší hrany je metoda váženého průměru stěn *Wall Average*, která pro určení orientace využívá informace o všech hranách budovy. Algoritmus vychází z předpokladu, že typické struktury budov jsou tvořeny primárně pravoúhlými stěnami. Získané úhly jednotlivých stěn jsou proto převedeny do prvního kvadrantu pomocí zbytku po dělení (modulo)

$pi/2$. Výsledný orientační směr budovy je pak určen jako průměr takto upravených úhlů, avšak s uplatněním délek jednotlivých stěn coby vah. Nejsilněji tedy výslednou orientaci ovlivňují ty nejdelší stěny, zatímco vliv drobných rohů a výstupků je minimalizován (Bayer 2026).

V naší implementaci je tento postup obsažen ve funkci `simplifyBuildingWallAverage`. Metoda využívá následující kroky:

1. **Výpočet délek a úhlů:** Algoritmus iteruje přes všechny hrany polygonu. Pro každou hranu se z rozdílu souřadnic (`dx`, `dy`) vypočte její délka (`length`) a její směrový úhel (`angle`) pomocí funkce `atan2`.
2. **Sjednocení orientace:** Pro sjednocení orientace vzájemně kolmých stěn se na vypočtený úhel aplikuje operace modulo $\pi/2$ (`angle % (pi/2)`).
3. **Aplikace vah:** Průběžně se sčítají celkové délky všech obvodových hran (`sum_lenghts`) a zároveň i součiny redukovaných úhlů s délkou dané hrany (`sum_angles_lenghts`).
4. **Výpočet váženého průměru:** Výsledný hlavní úhel budovy (`best_angle`) je následně vypočítán jako podíl sumy vážených úhlů a sumy délek.
5. **Opakující se kroky naší generalizace:** Se zjištěným hlavním úhlem se provedou totožné kroky tvorby obdélníku jako u ostatních metod:
   - Natočení polygonu budovy o úhel `-best_angle` (funkce `rotatePolygon`).
   - Vytvoření ohraničujícího *min-max boxu* (funkce `createMMB`).
   - Zpětná rotace obdélníku o úhel `best_angle`.
   - Proporční úprava plochy výsledného obdélníku tak, aby odpovídala ploše budovy (funkce `resizeRectangle`).

5.5 Algoritmus *Weighted Bisector*

Tento algoritmus přistupuje k hledání hlavní orientace budovy odlišným způsobem než předchozí metody. Namísto analýzy hran mnohouhelníku zkoumá diagonály mezi všemi vrcholy polygonu. Konkrétně vybírá dvě nejdelší různoběžné diagonály a výsledný orientační směr počítá jako jejich vážený průměr, přičemž jako váhy slouží právě délky těchto diagonál. Tím je zajištěno, že nejdelší osa budovy má na výsledek největší vliv (Bayer 2026).

V naší implementaci je tento postup obsažen ve funkci `simplifyBuildingWB`. Metoda využívá následující kroky:

1. **Generování diagonál:** Pomocí dvou cyklů se iteruje přes všechny dvojice nesousedních vrcholových bodů budovy. Pro každou platnou dvojici se z rozdílů souřadnic (`dx`, `dy`) vypočítá délka spojnice (`s`) a její směrový úhel (`sigma`). Úhel se následně redukuje do intervalu $< 0, \pi >$ a obě hodnoty se uloží do seznamu `diagonals`.

2. **Výběr primární diagonály:** Seznam vytvořených diagonál se seřadí sestupně podle jejich délky. První prvek seznamu je označen jako absolutně nejdelší diagonála s parametry délky `s1` a směrnice `sigma1`.

3. **Výběr sekundární diagonály:** Algoritmus dále prochází seřazený seznam a hledá druhou nejdelší diagonálu (`s_cand`, `sigma_cand`). Využívá přitom kontrolní podmínku: úhlový rozdíl (`diff`) mezi kandidátem a primární diagonálou musí být větší než námi stanovená tolerance (`angle_tolerance` = 10). Tím je zajištěno, že vybraná sekundární diagonála (`s2`, `sigma2`) není s tou primární rovnoběžná.

4. **Výpočet váženého směru:** Před samotným výpočtem průměru proběhne korekce úhlů, aby obě úsečky směřovaly do stejné poloroviny (rozdíl úhlů menší než $\pi/2$). Následně se vypočte výsledný orientační směr budovy (`sigma_weight`) podle rovnice váženého průměru:

$$\sigma_{weight} = \frac{s_1 \cdot \sigma_1 + s_2 \cdot \sigma_2}{s_1 + s_2}$$

5. **Opakující se kroky naší generalizace:** S vypočteným úhlem `sigma_weight` se provedou totožné kroky tvorby obdélníku jako u ostatních metod:
   - Natočení polygonu budovy o úhel `-sigma_weight` (funkce `rotatePolygon`).
   - Vytvoření ohraničujícího *min-max boxu* (funkce `createMMB`).
   - Zpětná rotace obdélníku o úhel `sigma_weight`.
   - Proporční úprava plochy výsledného obdélníku tak, aby odpovídala ploše budovy (funkce `resizeRectangle`).

6 Ošetření singulárních případů

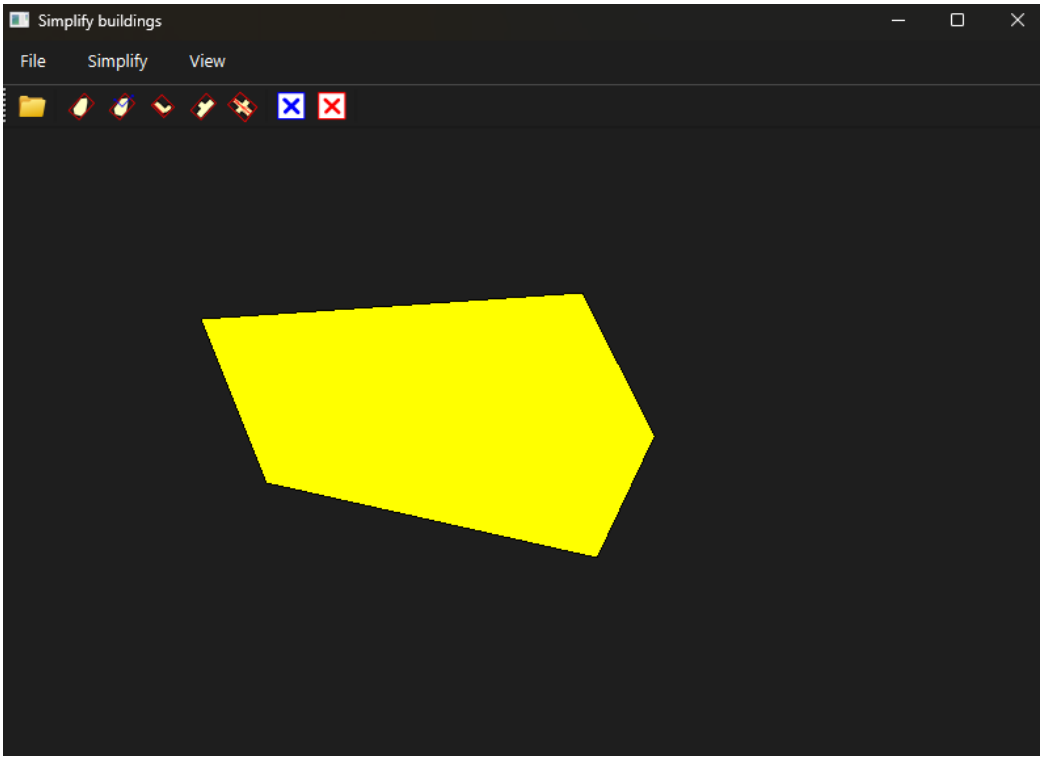
Tento bonusový úkol byl tvořen pro metodu tvorby konvexní obálky (*Jarvis Scan*), která je v naší implementaci řešena ve funkci `createCH` uvnitř třídy `Algorithms`. Bylo nutné ošetřit některé singulární případy, které se mohou v datech vyskytnout a způsobit pád aplikace nebo chybné vykreslení tvaru:

- **Nedostatečný počet bodů:** Zásadním krokem je kontrola vstupu. Pokud má vstupní množina bodů (polygon `pol`) méně než 3 vrcholy (`len(pol) < 3`), není možné konvexní obálku vytvořit a algoritmus by měl tyto body rovnou vrátit beze změn a bez dalšího výpočtu.
- **Inicializační vektor a prevence dělení nulou:** Algoritmus začíná od nejnižšího bodu (pivota `q`) a potřebuje referenční linii pro start měření úhlů k dalším bodům. Pokud by byl jako startovní pomocný bod zvolen bod se stejnými souřadnicemi jako pivot, vznikne vektor o nulové délce a výpočet ve funkci `get2VectorsAngle` spadne kvůli dělení nulou. Řešením je vytvoření umělého bodu `pj1` odsunutého po ose X vlevo od pivota s využitím třídy `QPointF(q.x() - 100, q.y())`, což zaručí vznik validního počátečního horizontálního vektoru.
- **Kolineární body:** Speciálním úkazem jsou body ležící na stejné přímce. V algoritmu to znamená, že testovaný kandidát svírá se současným vrcholem obálky identický úhel jako dosud nejlepší nalezený bod (`omega == omega_max`). Řešení spočívá ve výpočtu a porovnání druhé mocniny vzdáleností obou bodů od aktuálního vrcholu (`dist_max` a `dist_i`). Algoritmus ignoruje bližší body a aktualizuje index `index_max` pouze ve chvíli, kdy kandidát leží na přímce nejdále (`dist_i > dist_max`).

Pro úpravu některých částí kódu, zejm. syntaxe a chyb, byl použit nástroj AI. Tyto části kódu byly označeny komentáři, ve kterých je podrobněji popsáno, k čemu byla AI využita. Dále byla AI použita k nastínění postupu při tvorbě skriptu např. v hodnocení přesnosti (viz níže).

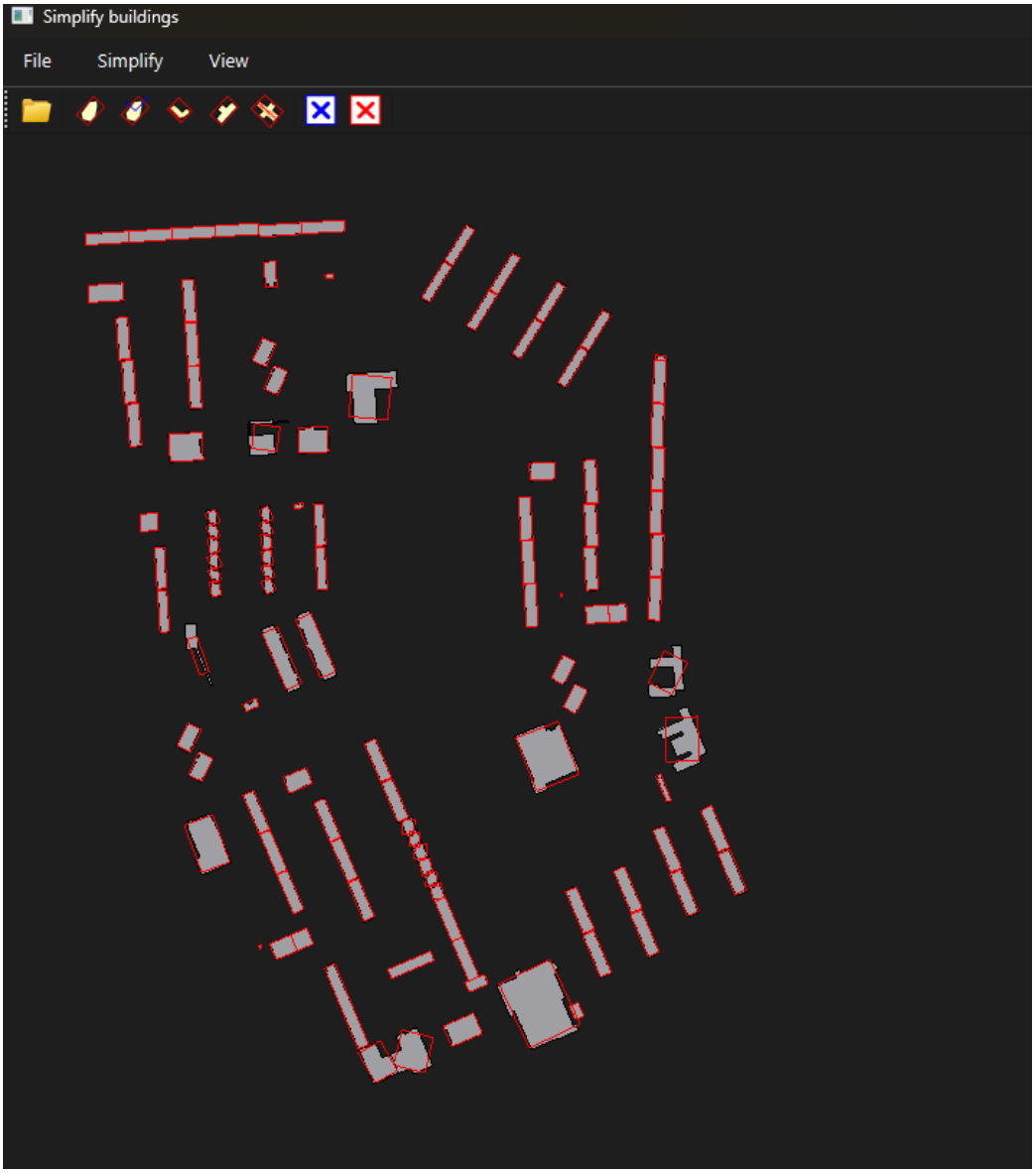
7 Popis aplikace

Pro generalizaci budov do úrovně detailu LOD0 byla vytvořena kostra aplikace v *QT Designer*, která byla dále upravena do výsledné podoby (obrázek2). Uživatel má možnost ručního kreslení polygonu pomocí klikání myši v okně či může zvolit možnost nahrání vlastního souboru *.shp* pomocí tlačítka *File -> Open* či ikonky složky.



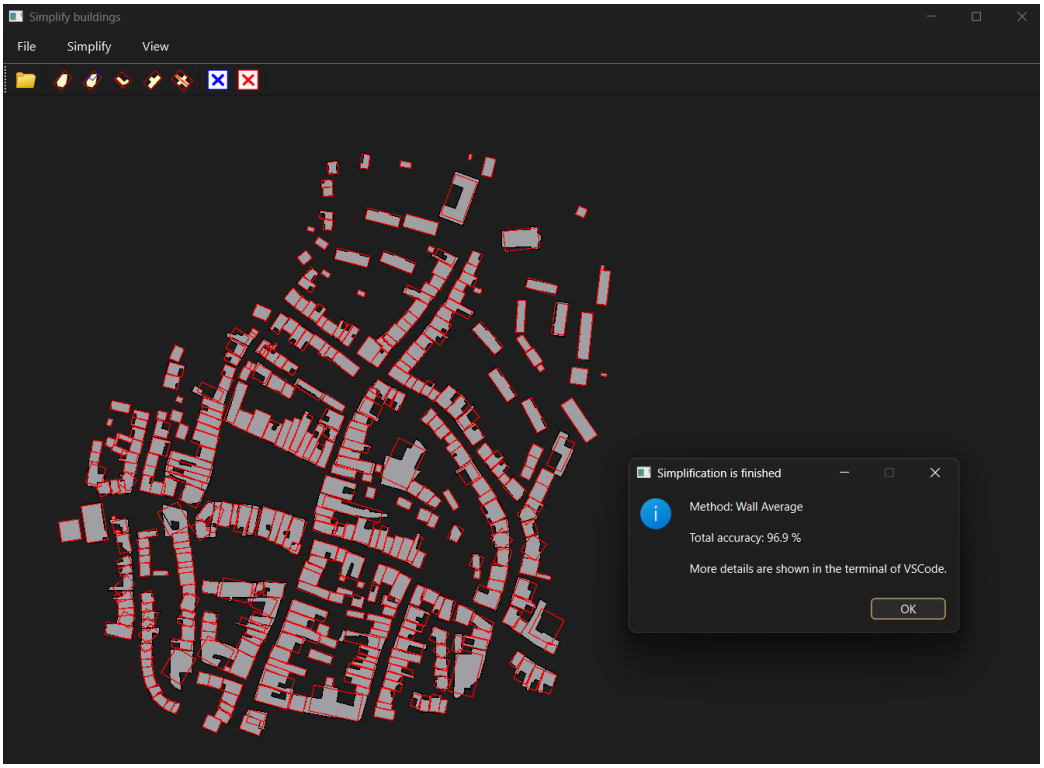
Obrázek 2: Okno aplikace s ručně vytvořeným polygonem

Pro generalizaci budovy si uživatel může zvolit z výše zmíněných algoritmů pomocí výběru ikonky z horní lišty. Po stisknutí zvolené ikonky proběhne výpočet generalizace a v okně se vykreslí červený obdélník pro každou budovu, který značí zjednodušený tvar (obrázek 3). Dále se také objeví vyskakovací okno s informacemi o použité metodě a její přesnosti (viz níže).



Obrázek 3: Generalizace sídliště Severní terasa pomocí algoritmu *Weighted Bisector*

Výstupem aplikace je kartografická generalizace polygonů v podobě jednoduchý obdélníků (LOD0), které mají stejnou plochu jako původní polygony a podrobný výpis přesnosti použité metody generalizace společně s výpisem počtu "správně"zjednodušených budov (obrázek 4).



Obrázek 4: Ukázka vyskakovacího okna s detaily generalizace

8 Hodnocení přesnosti

Jelikož jednotlivé metody generalizace fungují na lehce odlišných principech, lze očekávat, že jejich přesnost bude různá. Pro otestování a výpočet přesností byly jednotlivé metody použity na třech datasetech, které měly charakteristické půdorysy budov. První dataset je sídliště Severní terasa v Ústí n. Labem, ve které jsou především panelové domy, které mají většinou jednoduchý obdélníkový půdorys. Dataset sídliště obsahoval 113 budov.

Druhým datasetem je historické centrum v České Lípě, které je naopak charakteristické svými nepravidelnými a složitými půdorysy budov. Tento dataset obsahoval 418 budov.

Třetím datasetem je izolovaná vesnická zástavba obce Zákupy - Božíkov, ve které se objevují převážně izolované rodinné domy či chalupy. Tento dataset lze označit za komplexní, jelikož se zde objevují jak jednoduché tak i složitější půdorysy budov a obsahoval 107.

Výpočet byl prováděn pro jednotlivé budovy, za předpokladu, že jsou min-

max boxy pravoúhlé. Pro výpočet byla použita střední hodnota čtverců úhlových odchylek jednotlivých segmentů :

$$\Delta\sigma = \frac{\pi}{2n} \cdot \sqrt{\sum_{i=1}^{n} (r_i - \overline{r})^2}$$

kde byla zvolena hranice pro určení správnosti a akceptaci objektů hranice $\Delta\sigma < 10°$ (Bayer 2026).

Pro dataset sídliště, který obsahoval jednoduché půdorysy budov vyšly výsledky hodnocení přesnosti velice dobře. V tabulce 2, kde jsou zobrazené výsledky pro jednotlivé metody lze pozorovat, že se přesnosti pohybují nad 98 % u všech metod. Vysoká přesnost u tohoto datasetu je důsledkem velice jednoduchých půdorysů a tedy generalizace byla relativně jednoduchá. Díky tomu skoro všechny polygony LOD0 splňovaly výše zmíněnou podmínku 10°.

	sídliště (Ústí n. Labem)		
Metoda	**Počet budov**	**Správně**	**Přesnost** [%]
MBR	113	111	98.2
PCA	113	110	97.3
Longest Edge	113	111	98.2
Wall Avergae	113	111	98.2
Weighted Bisector	113	111	98.2

Tabulka 2: Hodnocení generalizace - sídliště

Dosti odlišným případem byl dataset historického centra v České Lípě. Z výsledků přesnosti zde vyčnívá metoda **PCA**, která dosahovala výrazně nižší přesnosti než zbylé metody (tabulka 3). Jelikož dataset obsahuje složité budovy, metoda PCA stěží určovala hlavní směry budovy a tudíž se nesplnila podmínka akceptace. Dále lze v tabulce 3 pozorovat, že se lehce snížila přesnost i ostatních metod na asi 96 %. K poklesu mohlo dojít i z důvodu většího počtu prvků než u ostatních datasetů.

	hist. centrum (Česká Lípa)		
Metoda	**Počet budov**	**Správně**	**Přesnost [%]**
MBR	418	407	97.4
PCA	418	370	88.5
Longest Edge	418	405	96.9
Wall Average	418	406	96.9
Weighted Bisector	418	401	95.9

Tabulka 3: Hodnocení generalizace - historické centrum

Posledním zkoumaným datasetem byla izolovaná zástavba. I v tomto případě vidíme v tabulce 4 relativně vysokou přesnost skoro všech metod, a to okolo 96-97 %. Opět však lze pozorovat výrazně nižší přesnost metody **PCA**, která je **86.9 %**. Přestože má datatset nejméně objektů, lze předpokládat, že obsahuje složité budovy (např. rodinné domy) a tudíž metoda PCA opět tyto objekty negeneralizuje správně.

	izolovaná zástavba (Zákupy - Božíkov)		
Metoda	**Počet budov**	**Správně**	**Přesnost [%]**
MBR	107	104	97.2
PCA	107	93	86.9
Longest Edge	107	104	97.2
Wall Average	107	103	96.3
Weighted Bisector	107	104	96.4

Tabulka 4: Hodnocení generalizace - izolovaná zástavba

9 Popis tříd a jejich metody

Pro aplikaci byly vytvořeny tři hlavní třídy. Třída **MainForm** má na starosti veškerou logiku, napojení tlačítek a načtení *.shp* souboru. Také je zde tvorba vyskakovacího okna s výsledky.

Pomocí třídy **Draw** jsou vykreslovány veškeré polygony, jak ručně, tak z *.shp* souboru.

Poslední třídou je **Algorithms**, ve které jsou obsaženy všechny metody generalizace a jejich matematický základ. Dále se zde nachází i výpočet přesnosti jednotlivých metod.

9.1 Třída *MainForm*

- **setupUi** - Inicializuje a rozmisťuje grafické prvky v okně aplikace a propojuje je s příslušnými funkcemi.
- **simplifyBuildingPCAClick** - Spouští generalizaci metodou ***PCA*** buď pro nakreslenou budovu, nebo pro všechny načtené polygony, provádí výpočet přesnosti a předává výsledky k vykreslení.
- **simplifyBuildingMBRClick** - Spouští generalizaci metodou ***MBR***.
- **simplifyBuildingWBClick** - Spouští generalizaci metodou ***Weighted Bisector***
- **simplifyBuildingLongestEdgeClick** - Spouští generalizaci metodou ***Longest Edge***.
- **simplifyBuildingWallAverageClick** - Spouští generalizaci metodou ***Wall Average***.
- **clearResultsClick** - Vymaže z plátna pouze vykreslené výsledky generalizace (červené obdélníky).
- **clearAllClick** - Kompletně vymaže veškerá data (výsledky, kreslenou budovu i načtené vrstvy ze *.shp*).
- **retranslateUi** - Nastavuje textové popisky a nápovědy u tlačítek v uživatelském rozhraní.
- **showEvaluationResults** - Zpracovává statistiku generalizace. Vypočítá procentuální úspěšnost (odchylka do 10°), vypíše detaily do terminálu a zobrazí vyskakovací okno s výsledkem.
- **openFile** - Otevírá okno s cestou na disku k volbě **.shp* souboru.
- **loadShapefile** - Načte vybraný **.shp* soubor, vypočítá *min-max box* celého datasetu pro určení správného měřítka zobrazení a převede souřadnice do seznamu polygonů.

9.2 Třída *Algorithms*

- **get2VectorsAngle** - Vypočítá a vrátí úhel mezi dvěma zadanými vektory definovanými čtyřmi body.
- **createCH** - Vytvoří konvexní obálku nad množinou bodů polygonu pomocí algoritmu *Jarvis Scan*, včetně ošetření singulárních případů.

- **createMMB** - Vytvoří pravoúhlý *min-max box* rovnoběžný s osou x a vypočte jeho plochu.
- **rotatePolygon** - Zrotuje všechny body zadaného polygonu o vypočítaný úhel $-\sigma$.
- **createMBR** - Nalezne ohraničující obdélník s minimální plochou pomocí iterativního otáčení konvexní obálky.
- **getArea** - Vypočítá celkovou plochu zadaného polygonu.
- **resizeRectangle** - Upraví měřítko výsledného obdélníku z jeho těžiště tak, aby jeho plocha odpovídala ploše původní budovy.
- **simplifyBuildingMBR** - Generalizace metodou ***MBR***.
- **simplifyBuildingPCA** - Generalizace metodou ***PCA***.
- **simplifyBuildingWB** - Generalizace metodou ***Weighted Bisector***.
- **simplifyBuildingLongestEdge** - Generalizace metodou ***Longest Edge***.
- **simplifyBuildingWallAverage** - Generalizace metodou ***Wall Average***.
- **getDirection** - Vypočte směrový úhel hrany definované dvěma body pomocí funkce *atan2*.
- **ResultsEvaluation** - Vyhodnotí přesnost generalizace. Počítá průměrnou úhlovou odchylku reálných stěn budovy od vypočteného hlavního směru LOD0 a vrací výsledek ve stupních.

9.3 Třída *Draw*

- **mousePressEvent** - Zaznamenává kliknutí myší do plátna a přidává nové lomové body do polygonu ručně kreslené budovy.
- **paintEvent** - Hlavní vykreslovací metoda. Zajišťuje zobrazení načtených shapefilů či ručně kreslené budovy a výsledné generalizované obdélníky.
- **setMBR / setMBRs** - Uloží výsledný obdélník (nebo seznam obdélníků) pro vykreslení.
- **setCH** - Uloží vypočítanou konvexní obálku.
- **getBuilding** - Vrací polygon ručně nakreslené budovy.
- **clearResult** - Vymaže pouze výsledky generalizace.

- **clearData** - Vymaže vše a nastaví výsledky do původního stavu.
- **getPols** - Vrací seznam všech existujících polygonů.
- **setPols** - Nahradí aktuální polygony na plátně novými, vybranými ze *.shp*

10 Závěr

Úkolem této úlohy bylo vytvořit aplikaci pro generalizaci mnohoúhelníků a budov na úroveň LOD0. Pro generalizaci bylo použito pět metod: *Minimum Area Enclosing Rectangle*, *Principal Component Analysis*, *Longest Edge*, *Wall Average* a *Weighted Bisector*. Každá z nich je založena na jiném principu hledání hlavního směru polygonu a proto se i jejich výsledné generalizace liší. Všechny metody byly velice spolehlivé až na metodu **PCA**, která dosahovala nižších přesností v závislosti od zadané úhlovíé odchylky.

Možnost ručně nakresleného polygonu či nahrání souboru *.shp* je dobrá volba pro případné lokální analýzy. Nevýhodou je absence přibližování či posunování v "mapovém"okně.

Výhodou aplikace je možnost přidání dalších metod a průběžné vylepšování či dolaďování uživatelského rozhraní v jazyce Python.

Zdroje

Bayer, Tomáš (2026). *Prezentace Agoritmů počítačové kartografie, Konvexní obálka množiny bodů.* URL: `https://web.natur.cuni.cz/~bayertom/images/courses/Adk/adk4_new.pdf`.

OpenGenusIQ (2026). *Gift Wrap Algorithm (Jarvis March Algorithm) to find Convex Hull.* URL: `https://iq.opengenus.org/gift-wrap-jarvis-march-algorithm-convex-hull/`.